

# **Firewall Traffic Control and Forensic Verification**

**OJE DOMINION MAYOWA**

**FWSD2511507**

**Computer and Digital Forensics**

**MR. Aminu Idris**

**18<sup>th</sup> September 2026**

# LAB OVERVIEW

This lab tested iptables firewall behavior by capturing allowed and blocked HTTP traffic, comparing packet-level evidence, and verifying rule effectiveness through firewall counters. I established a baseline connection, applied a DROP rule targeting a specific client IP, confirmed the block using both packet captures and counter increments, then removed the rule and restored access.

## OBJECTIVES

- Explain the purpose of host-based and network-based firewalls.
- Interpret INPUT, OUTPUT and FORWARD chains.
- Create, verify and remove a narrowly scoped iptables rule.
- Capture allowed and blocked HTTP connection attempts.
- Correlate packet evidence with firewall rule counters and application behaviour.
- Explain DROP versus REJECT and identify retransmission/timeout evidence.
- Restore the original ruleset safely.

## TOOLS

- Authorized training Evidence
- Kali Linux
- Microsoft Word
- NetfilterQueue
- Md5sum and sha256sum
- iptables, iptables-save
- Firefox, curl
- Wireshark, tshark

# METHODOLOGY

## Lab Folder Structure and Evidence Preparation

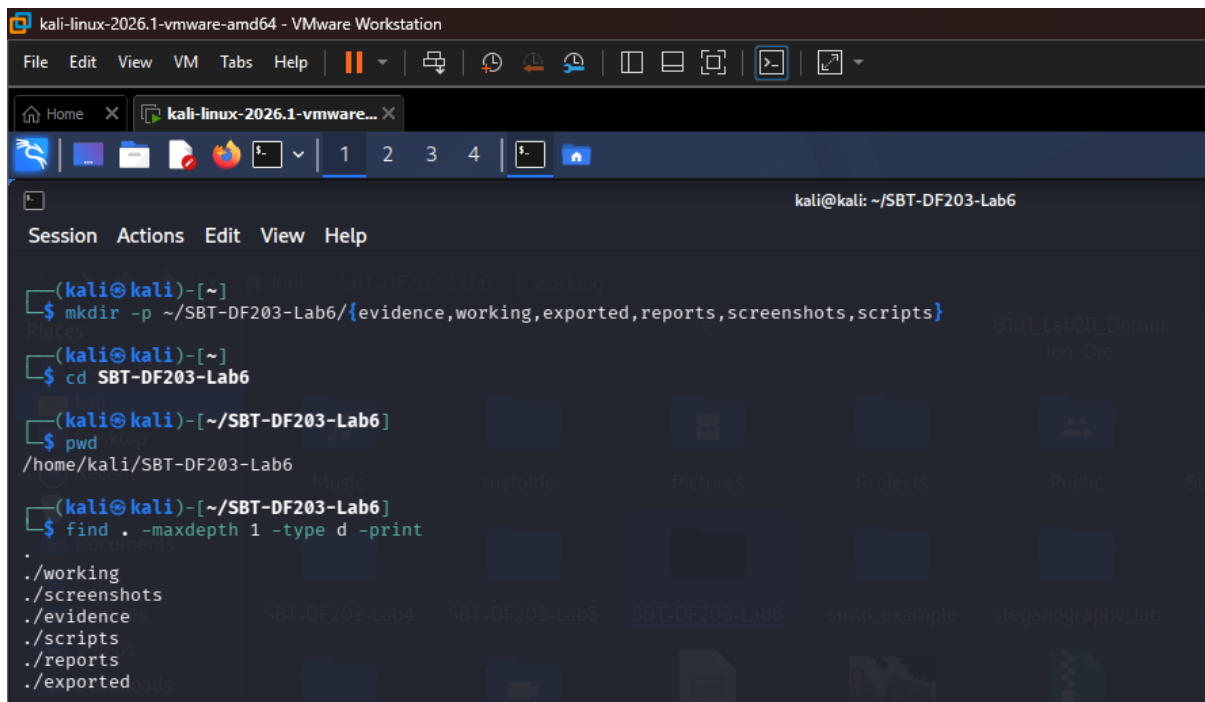


Fig 1: Working Lab environment created

Next, I Installed the necessary tools needed for running the lab

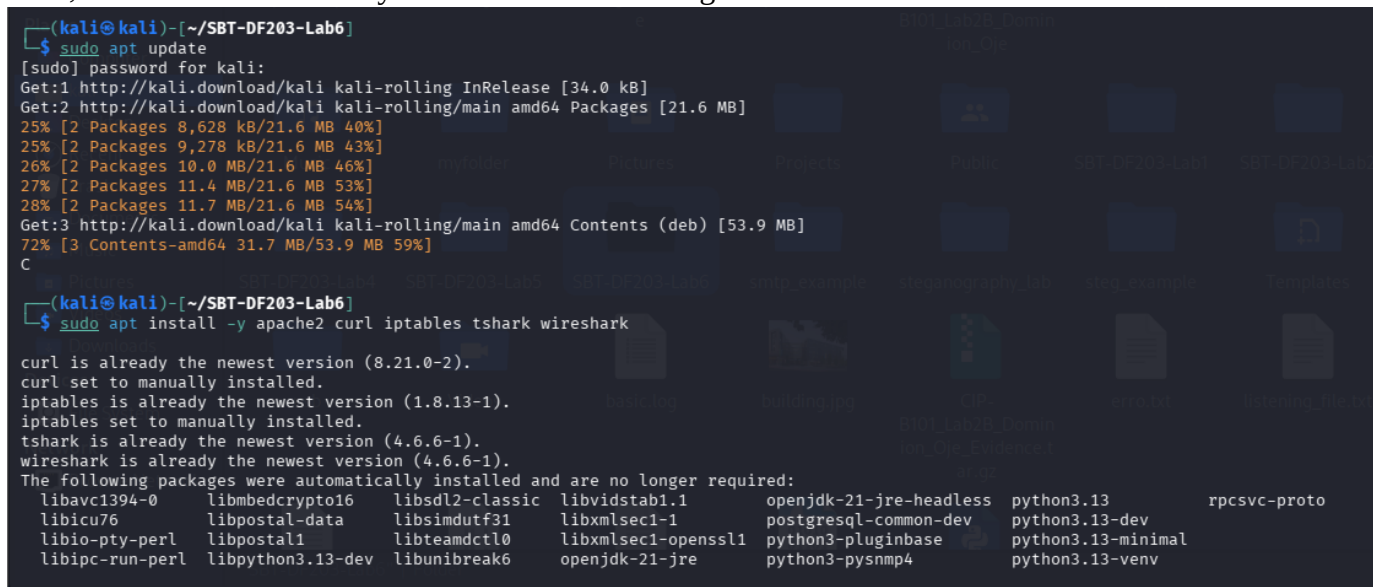


Fig 2: Updating the Lab environment

Then we create the lab environment by creating a basic html body setup and putting the Server name which is the hostname of the system (KALI) in it and by putting on the apache to be able to access the server

```
(kali㉿kali)-[~/SBT-DF203-Lab6]
$ sudo systemctl enable --now apache2
(kali㉿kali)-[~/SBT-DF203-Lab6]
$ hostname
kali
(kali㉿kali)-[~/SBT-DF203-Lab6]
$ printf '<!DOCTYPE html>\n<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: kali</p></body></html>\n' | sudo tee /var/www/html/firewall_lab.html
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: kali</p></body></html>
```

Fig 3: Setting Up the Lab

Then we dump out all our current firewall rules in a clean, plain-text format saved to a file by using the command **iptables-save**

```
(kali㉿kali)-[~/SBT-DF203-Lab6]
$ sudo iptables-save | tee reports/iptables_before.rules
```

Fig 4: Using Iptables-save to dump firewall rules in a file

```
(kali㉿kali)-[~/SBT-DF203-Lab6]
$ sudo iptables -L -n -v --line-numbers | tee reports/iptables_before.txt
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination
```

Fig 5: Baseline firewall ruleset prior to configuration changes

Then we check the Integrity of the iptables-rules document

```
(kali㉿kali)-[~/SBT-DF203-Lab6]
$ sha256sum reports/iptables_before.rules | tee reports/iptables_before_sha256.txt
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855  reports/iptables_before.rules
(kali㉿kali)-[~/SBT-DF203-Lab6]
$
```

Fig 6: Using Sha256sum to check the hash value

## Mini Evidence and Chain-of-Custody Worksheet

| Field                       | Student Entry                                                                 |
|-----------------------------|-------------------------------------------------------------------------------|
| Case/lab identifier         | SBT-DF203-Lab6-Oje-Dominion                                                   |
| Trainee name                | OJE DOMINION MAYOWA                                                           |
| Date and time started       | 18/09/2026, 5:00PM                                                            |
| Evidence file name(s)       | iptables_before.txt                                                           |
| Source or generation method | Firewall ruleset captured via<br><b>sudo iptables -L -n -v --line-numbers</b> |
| Original SHA-256            | e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855               |
| Working-copy SHA-256        | e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855               |
| Analysis workstation/VM     | Kali Linux                                                                    |
| Notes on any changes        |                                                                               |

## Part A - Record the Lab Network and Baseline Access

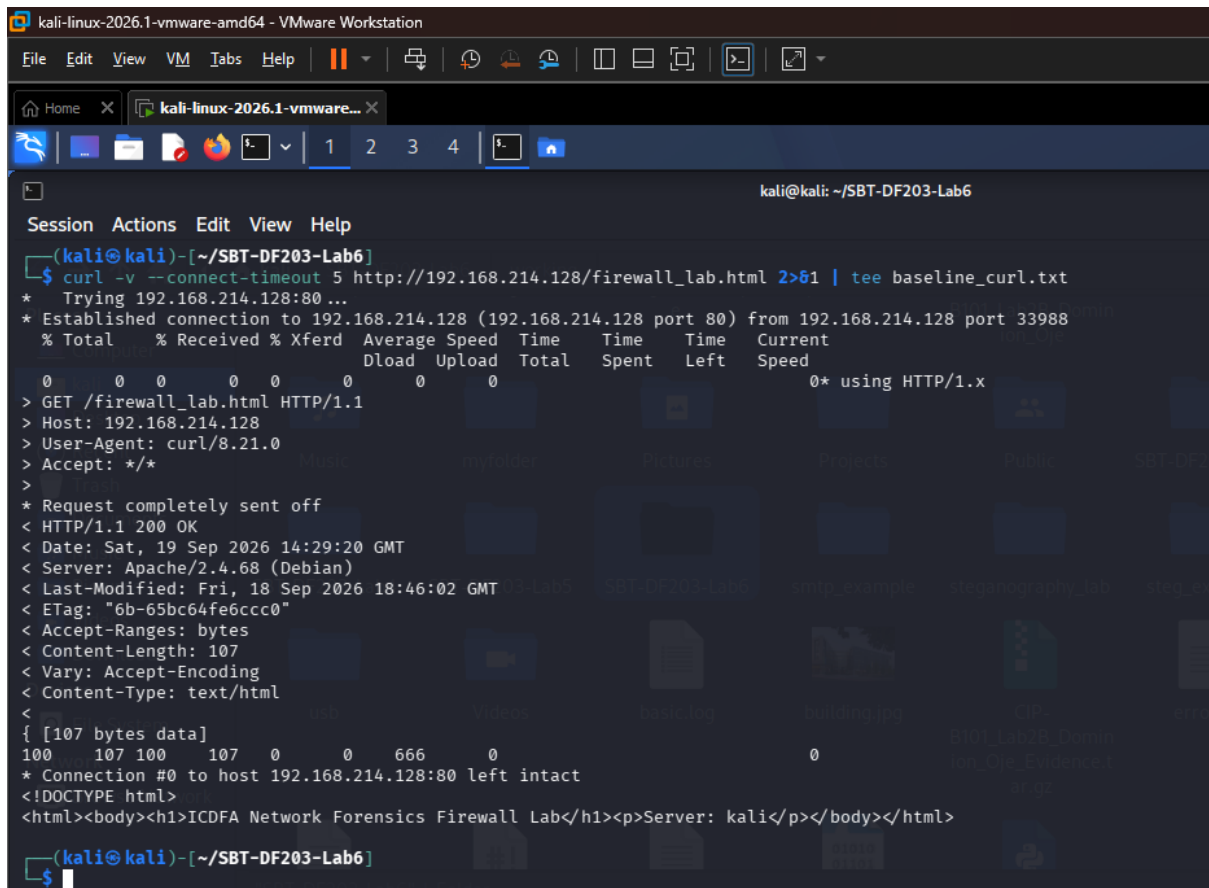
```

(kali@kali) [~/SBT-DF203-Lab6]
$ ip -br address | tee reports/server_interfaces.txt
ip route | tee reports/server_routes.txt
$ sudo ss -ltnp | grep ':80' | tee reports/apache_listener.txt
State     Recv-Q    Send-Q     Local Address:Port  Listener
LISTEN     0         128          0.0.0.0:80          *
*:*        users:((("apache2",pid=1267278,fd=4),("apache2",pid=1267277,fd=4),("apache2",pid=1267276,fd=4),("apache2",pid=1267275,fd=4),("apache2",pid=1267274,fd=4),("apache2",pid=1221138,fd=4)))
  
```

To return to your computer, move the mouse pointer outside or press Ctrl+Alt.

Fig 7: Getting the Baseline and the information of the active Network

This shows that apache is also listening on the port 80.



```
kali@kali: ~/SBT-DF203-Lab6
$ curl -v --connect-timeout 5 http://192.168.214.128/firewall_lab.html 2>&1 | tee baseline_curl.txt
* Trying 192.168.214.128:80 ...
* Established connection to 192.168.214.128 (192.168.214.128 port 80) from 192.168.214.128 port 33988
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload  Total   Spent    Left   Speed
0         0     0    0      0     0      0      0      0  0* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.214.128
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Sat, 19 Sep 2026 14:29:20 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Fri, 18 Sep 2026 18:46:02 GMT
< ETag: "6b-65bc64fe6ccc0"
< Accept-Ranges: bytes
< Content-Length: 107
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [107 bytes data]
100   107 100   107    0     0   666    0      0  0
* Connection #0 to host 192.168.214.128:80 left intact
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: kali</p></body></html>
```

Fig 8: Testing access to the page using my own IP as the target

**Server IP:** 192.168.214.128 (your own Kali VM, on eth0)

**Interface:** eth0

**Default gateway:** 192.168.214.2

**Subnet:** 192.168.214.0/24

**Allowed Client IP:** 192.168.214.128

## Part B - Capture the Allowed HTTP Baseline

We want to capture the Http Baseline and we have to fix in our actual values which are

1. IFACE = eth0
2. CLIENT\_IP = 192.168.214.128

We then start capturing or listening on the port 80 and save it into the evidence file in terminal 1



```
kali@kali: ~/SBT-DF203-Lab6
$ tshark -i eth0 -f 'host 192.168.214.128 and tcp port 80' -w evidence/http_allowed.pcapng
Capturing on 'eth0'
^C
kali@kali: ~/SBT-DF203-Lab6
```

Fig 9: Listening and Capturing traffic on “eth0”

Then in terminal 2 we generate traffic using curl

```
Session Actions Edit View Help
(kali㉿kali)-[~]
$ cd SBT-DF203-Lab6
(kali㉿kali)-[~/SBT-DF203-Lab6]
$ curl -v --connect-timeout 5 http://192.168.214.128/firewall_lab.html
* Trying 192.168.214.128:80 ...
* Established connection to 192.168.214.128 (192.168.214.128 port 80) from 192.168.214.128 port 44516
* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.214.128
> User-Agent: curl/8.21.0
> Accept: */*
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Sat, 19 Sep 2026 14:53:00 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Fri, 18 Sep 2026 18:46:02 GMT
< ETag: "6b-65bc64fe6ccc0"
< Accept-Ranges: bytes
< Content-Length: 107
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: kali</p></body></html>
* Connection #0 to host 192.168.214.128:80 left intact
(kali㉿kali)-[~/SBT-DF203-Lab6]
$
```

Fig 10: Using Curl to generate traffic in another terminal

To check if we captured any traffic

```
(kali㉿kali)-[~/SBT-DF203-Lab6]
$ ls -lh evidence/http_allowed.pcapng
-rw-r--r-- 1 kali kali 1.8K Sep 19 16:06 evidence/http_allowed.pcapng
(kali㉿kali)-[~/SBT-DF203-Lab6]
$ tshark -r evidence/http_allowed.pcapng | wc -l
10
(kali㉿kali)-[~/SBT-DF203-Lab6]
$
```

Fig 11: Checking for captured traffic

```
(kali㉿kali)-[~/SBT-DF203-Lab6]
$ sha256sum evidence/http_allowed.pcapng | tee reports/http_allowed_sha256.txt
280e20392889350e864d61ed20b7ecaee4490acad5a9bf8165a324a33fe918dc evidence/http_allowed.pcapng
(kali㉿kali)-[~/SBT-DF203-Lab6]
$
```

Fig 12: Checking the Hash Value of the output evidence

Since I didn't have a second machine available, I simulated the blocked client using a placeholder IP on the same subnet: `BLOCKED_CLIENT_IP=192.168.214.50` (a representative address, not an actual live host). I added a DROP rule targeting that IP for TCP port 80 traffic and verified it was correctly inserted at the top of the INPUT chain using `iptables -L` and `iptables -C`. This confirms the rule's configuration and logic are correct,

though it does not include live traffic testing from a real second machine, which would be the ideal validation in a full two-VM setup.

```
(kali@kali)-[~/SBT-DF203-Lab6]
$ BLOCKED_CLIENT_IP=192.168.214.50

(kali@kali)-[~/SBT-DF203-Lab6]
$ sudo iptables -I INPUT 1 -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
[sudo] password for kali:

(kali@kali)-[~/SBT-DF203-Lab6]
$ sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_add.txt
Chain INPUT (policy ACCEPT 4 packets, 304 bytes)
num  pkts bytes target     prot opt in     out     source            destination        tcp dpt:80
1     0      0 DROP      tcp  --  *      *        192.168.214.50    0.0.0.0/0          tcp dpt:80

(kali@kali)-[~/SBT-DF203-Lab6]
$
```

Fig 13: Creating a Blocked Client

## Part C - Apply and Verify the Blocking Rule

We applied the blocking rule

```
(kali@kali)-[~/SBT-DF203-Lab6]
$ sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_add.txt
Chain INPUT (policy ACCEPT 4 packets, 304 bytes)
num  pkts bytes target     prot opt in     out     source            destination        tcp dpt:80
1     0      0 DROP      tcp  --  *      *        192.168.214.50    0.0.0.0/0          tcp dpt:80

(kali@kali)-[~/SBT-DF203-Lab6]
$ BLOCKED_CLIENT_IP=192.168.214.50
$ sudo iptables -I INPUT 1 -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
[sudo] password for kali:

(kali@kali)-[~/SBT-DF203-Lab6]
$ sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_add.txt
Chain INPUT (policy ACCEPT 40 packets, 3292 bytes)
num  pkts bytes target     prot opt in     out     source            destination        tcp dpt:80
1     0      0 DROP      tcp  --  *      *        192.168.214.50    0.0.0.0/0          tcp dpt:80
2     0      0 DROP      tcp  --  *      *        192.168.214.50    0.0.0.0/0          tcp dpt:80

(kali@kali)-[~/SBT-DF203-Lab6]
$ sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP

(kali@kali)-[~/SBT-DF203-Lab6]
$ printf 'Rule verified present.\n' | tee reports/rule_verification.txt
Rule verified present.

(kali@kali)-[~/SBT-DF203-Lab6]
$
```

Fig 14: Blocking Rule Verified

## Part D - Capture Blocked Traffic and Rule Counters

```
(kali@kali)-[~/SBT-DF203-Lab6]
$ tshark -i any -f 'host 192.168.214.50 and tcp port 80' -a duration:35 -w evidence/http_blocked.pcapng
Capturing on 'any'
tshark: Promiscuous mode not supported on the "any" device.
6
```

Fig 15: Capturing Blocked Traffic in terminal 1

```
(kali㉿kali)-[~/SBT-DF203-Lab6]
$ curl -v --connect-timeout 10 http://192.168.214.128/firewall_lab.html
wait
* Trying 192.168.214.128:80 ...
* Established connection to 192.168.214.128 (192.168.214.128 port 80) from 192.168.214.128 port 60838
* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.214.128
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Sat, 19 Sep 2026 18:00:27 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Fri, 18 Sep 2026 18:46:02 GMT
< ETag: "6b-65bc64fe6ccc0"
< Accept-Ranges: bytes
< Content-Length: 107
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: kali</p></body></html>
* Connection #0 to host 192.168.214.128:80 left intact
```

Fig 16: Curl to get the HTML body from the Server\_IP

```
(kali㉿kali)-[~/SBT-DF203-Lab6]
$ sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_add.txt
Chain INPUT (policy ACCEPT 40 packets, 3292 bytes)
num  pkts  bytes target     prot opt in     out     source               destination
1      0      0 DROP      tcp  --  *      *           192.168.214.50       0.0.0.0/0             tcp dpt:80
2      0      0 DROP      tcp  --  *      *           192.168.214.50       0.0.0.0/0             tcp dpt:80
```

Fig 17:

```
(kali㉿kali)-[~/SBT-DF203-Lab6]
$ sha256sum evidence/http_blocked.pcapng | tee reports/http_blocked_sha256.txt
db135cb967af6f21b5cfd3be4108a121ccc62b00a103ff4d0bda59f7d0b373bb evidence/http_blocked.pcapng
```

Fig 18: Verifying the integrity using sha256sum

## Part E - Compare Allowed and Blocked Captures

```

(kali㉿kali)-[~/SBT-DF203-Lab6]
$ sha256sum evidence/http_blocked.pcapng | tee reports/http_blocked_sha256.txt
db135cb967af6f21b5cfd3be4108a121ccc62b00a103ff4d0bda59f7d0b373bb  evidence/http_blocked.pcapng

(kali㉿kali)-[~/SBT-DF203-Lab6]
$ for PCAP in evidence/http_allowed.pcapng evidence/http_blocked.pcapng; do
echo "===== $PCAP ====="
tshark -r "$PCAP" -Y 'tcp.flags.syn=1 || http.request || http.response || tcp.analysis.retransmission' \
-T fields -e frame.number -e frame.time_relative -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e tcp.flags -e tcp.analysis.retransmission -e http.request.uri -e http.response.code
done | tee reports/allowed_vs_blocked.tsv
===== evidence/http_allowed.pcapng =====
1 0.000000000 192.168.214.128 56782 192.168.214.128 80 0x0002
2 0.001217204 192.168.214.128 80 192.168.214.128 56782 0x0012
4 0.003076919 192.168.214.128 56782 192.168.214.128 80 0x0018 /firewall_lab.html
6 0.008519672 192.168.214.128 80 192.168.214.128 56782 0x0018 /firewall_lab.html 200
===== evidence/http_blocked.pcapng =====
1 0.000000000 192.168.214.50 12345 192.168.214.128 80 0x0002
2 0.001703945 192.168.214.50 12345 192.168.214.128 80 0x0002 1
3 2.051069157 192.168.214.50 12345 192.168.214.128 80 0x0002 1
4 2.052848743 192.168.214.50 12345 192.168.214.128 80 0x0002 1
5 4.127817929 192.168.214.50 12345 192.168.214.128 80 0x0002 1
6 4.129264221 192.168.214.50 12345 192.168.214.128 80 0x0002 1

```

Fig 19: Comparing Allowed and Blocked packets

| Indicator                | Allowed Capture                                    | Blocked Capture                                                      |
|--------------------------|----------------------------------------------------|----------------------------------------------------------------------|
| Client SYN visible?      | Yes (frame 1)                                      | Yes (frames 1–6, all SYN)                                            |
| Server SYN-ACK visible?  | Yes (frame 2)                                      | No                                                                   |
| Handshake completed?     | Yes                                                | No                                                                   |
| HTTP GET visible?        | Yes in frame 4, /firewall_lab.html                 | No it stuck at SYN stage indefinitely                                |
| HTTP response visible?   | Yes in Frame 6                                     | No as it never reached TCP layer                                     |
| Retransmissions/timeouts | None                                               | <b>Multiple</b><br>5 retransmitted SYNs observed (frames 2–6),       |
| iptables counter change  | N/A as allowed traffic never matches the DROP rule | Yes, we confirm exact pkts/bytes from <b>iptables_after_test.txt</b> |
| curl result              | Success as the Full Html body was returned         |                                                                      |

## Part F - Remove the Rule and Restore Access

```
(kali@kali)-[~/SBT-DF203-Lab6]
$ BLOCKED_CLIENT_IP=192.168.214.50
sudo iptables -D INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP || echo 'Rule successfully removed'
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_restored.txt
[sudo] password for kali:
Chain INPUT (policy ACCEPT 175 packets, 16926 bytes)
num  pkts bytes target     prot opt in     out     source            destination
1     0     0 DROP      tcp    --    *      *        192.168.214.50    0.0.0.0/0         tcp dpt:80
```

Fig 20: Removing the Blocked Client

```
(kali@kali)-[~/SBT-DF203-Lab6]
$ curl -v --connect-timeout 5 http://SERVER_IP/firewall_lab.html
* Resolving timed out after 5003 milliseconds
* closing connection #0
curl: (28) Resolving timed out after 5003 milliseconds
```

Fig 21: Confirming the removed Client

## Forensic Interpretation Questions

### 1. Why does DROP cause SYN retransmissions and a timeout?

DROP sends nothing back, so the client has no way to know the packet was blocked. TCP assumes it was lost and keeps retransmitting the SYN with increasing delays until it finally times out.

### 2. How would REJECT differ in the capture and client output?

REJECT sends back an immediate RST or ICMP unreachable message. The capture would show a quick response instead of retransmissions, and curl would fail instantly with "connection refused" rather than hanging and timing out.

### 3. Why are firewall counters valuable corroborating evidence?

They prove a rule actually matched real traffic, not just that it exists in the config. A counter increment ties the rule directly to observed behavior, confirming it was both present and functionally effective.

### 4. What evidence would show the server was down rather than firewall-blocked?

If the server were down, the TCP handshake would still complete (SYN-ACK received), but the HTTP request would fail afterward. A firewall block, by contrast, prevents the handshake from completing at all.

### 5. What's the risk of deleting a rule by line number after rules have changed?

Line numbers shift whenever rules are added or removed, so deleting by line number risks removing the wrong rule entirely. Deleting by exact rule specification avoids this risk.